



Department of Computer Science

ASSIGNMENT 2

Course: Object Oriented Programming

Teacher: Sir Zulqurnain

Section: BS(AI) 2A, 2B

Instructions:

- Do not share your code with anyone else. All assignments are to be done individually.
- **Plagiarized Solutions** of any type will lead to **ZERO marks**.
- Submit only your source code `.cpp` file. Name your file as `rollno_name_dept_sec.cpp` (e.g., `P22-1234_Name_AI_2A.cpp`).
- Follow the naming scheme correctly; failure will result in **ZERO marks**.
- Submissions only through respective **Google Classroom** will be entertained.
- Make sure your code **compiles and runs**; non-compiling code will get **ZERO marks**.
- You are **not** allowed to use STL containers (`vector`, `list`, etc.). Use raw arrays only.
- **Late submissions** will not be entertained.
- If there is confusion, ask the TA or make reasonable assumptions, but only those that do not conflict with the question.
- Eid Mubarak — Happy Coding!

Question 1: Eid ul Fitr Mela Management System

Scenario

The crescent moon has been sighted and **Eid ul Fitr** has arrived in the bustling city of **Al-Noor**! Every year the city celebrates with a grand **Eid Mela** — a festival full of sweet stalls, excited visitors, Eidi distributions, and joyful families dressed in new clothes.

This year the **Al-Noor City Council** has decided to go digital. They need a **C++ Object-Oriented system** to manage the entire Eid Mela electronically: tracking the fair grounds, the sweet stalls, and all visiting families. You have been hired as the **lead developer**. Your job is to design and implement the complete system using the three classes described below.

Class 1: EidMela

“The grand fair grounds of Al-Noor City. The city council tracks every mela ever held.”

Static Members:

- `static int totalMelasHeld`
- `static int totalStallsAcrossAllMelas`

Data Members:

- `string melaName`
- `string location`
- `int totalStalls`
- `double entryFee`
- `bool isOpen`

Constructors & Destructor:

- **Default constructor:** Set `melaName = "Unnamed Mela"`, `location = "TBD"`, `totalStalls = 0`, `entryFee = 0`, `isOpen = false`. Increment `totalMelasHeld`.
- **Parameterized constructor:** Accept `melaName`, `location`, `totalStalls`, `entryFee`. Set `isOpen = true`. Increment `totalMelasHeld`. Add `totalStalls` to `totalStallsAcrossAllMelas`.
- **Destructor:** Set `isOpen = false`. Print, e.g.:
[Mela Closed] Al-Noor Grand Eid Mela has ended. Khuda Hafiz!

Class 2: SweetStall

“Eid without mithai is unimaginable. Each stall carries its own inventory of Eid sweets. When a stall closes, its sweets are gone too.”

Static Members:

- `static int totalSweetStalls`
- `static double totalRevenueAllStalls`

Data Members:

- `string stallName`
- `int stallID` — auto-assigned using a static counter starting from 101
- `string* sweetItems` — **dynamically allocated** array of sweet names (e.g., Sheer Khurma, Sewaiyan, Barfi)
- `int itemCount`
- `int maxItems`
- `double pricePerKg`
- `double totalKgSold`

Constructors & Destructor:

- **Default constructor:** `stallName = "Unknown Stall"`, auto-assign `stallID`, allocate `sweetItems` with `maxItems = 3`, `pricePerKg = 0`, `totalKgSold = 0`. Increment `totalSweetStalls`.
- **Parameterized constructor:** Accept `stallName`, `maxItems`, `pricePerKg`. Auto-assign `stallID`. Allocate `sweetItems`. Increment `totalSweetStalls`.
- **Copy constructor:** Perform a **deep copy** of the `sweetItems` array. Assign a **new unique** `stallID` (do NOT copy the original's ID). Increment `totalSweetStalls`.
- **Destructor:** `delete[] sweetItems`. Decrement `totalSweetStalls`. Print, e.g.:
[Stall Closed] Karachi Mithai Wala (ID: 101) has packed up. Ma'assalam!

Class 3: Visitor

“Families pour into the mela wearing new clothes, excited children in tow. Each visitor receives Eidi, buys sweets, and creates memories.”

Static Members:

- `static int totalVisitors`
- `static double totalEidiDistributed`

Data Members:

- `string name`
- `int visitorID` — auto-incremented from 5001
- `int age`
- `double eidiReceived`
- `double budget`
- `string* purchasedItems` — **dynamically allocated** array of purchased item names
- `int purchaseCount`
- `int maxPurchases`

Constructors & Destructor:

- **Default constructor:** `name = "Guest"`, auto-assign `visitorID`, `age = 0`, `eidiReceived = 0`, `budget = 500`. Allocate `purchasedItems` with `maxPurchases = 5`. Increment `totalVisitors`.
- **Parameterized constructor:** Accept `name`, `age`, `budget`, `maxPurchases`. Allocate `purchasedItems`. Auto-assign `visitorID`. Increment `totalVisitors`.
- **Copy constructor:** Perform a **deep copy** of `purchasedItems`. Assign a new unique `visitorID`. Increment `totalVisitors`.
- **Destructor:** `delete[] purchasedItems`. Decrement `totalVisitors`. Print, e.g.:
[Visitor Left] Ahmed has left the mela. Eid Mubarak!

What to do in main()

1. Create **2 EidMela** objects (one using default constructor, one using parameterized constructor). Display their info and demonstrate `totalMelasHeld` updating correctly.
2. Create **3 SweetStall** objects with different sweet inventories and prices. Make a **copy** of one stall using the copy constructor and verify it gets a new unique `stallID`.
3. Create **4 Visitor** objects (mix of default and parameterized). Give each visitor some Eidi by directly updating `eidiReceived`, `totalEidiDistributed`, and `budget`. Make a **copy** of one visitor and verify deep copy works (modifying the copy must not affect the original).
4. Place **1 stall** and **1 visitor** inside a nested scope block `{ }`. After the block ends, verify that their destructor messages printed and `totalSweetStalls` / `totalVisitors` decremented correctly.
5. At the end of `main()`, print the final city-wide statistics using the static members: `totalMelasHeld`, `totalStallsAcrossAllMelas`, `totalVisitors`, and `totalEidiDistributed`.